OptMetTools

Andrew Humphreys

Jul 29, 2026

CONTENTS:

1 Interferogram 1

2 Surface 4

Index 8

CHAPTER

ONE

INTERFEROGRAM

This module contains methods relavent to simulation and processing WLI and PSI interferograms.

class OptMetTools.Interferogram.process.**Process**

Bases: object

Define several algorithms for processing interferograms.

Provides vectorized implementation, allowing for processing of many unrelated interferogram stacks.

References:

1. https://doi.org/10.1364/OE.27.037634
2. https://doi.org/10.1016/j.optlaseng.2021.106768
3. https://doi.org/10.1088/2040-8978/17/2/025704
4. http://doi.org/10.1080/09500349514550341
5. https://doi.org/10.1364/OL.29.001671
6. https://doi.org/10.1016/j.optlaseng.2005.11.003

static AIA(*I*, *delta*, *epsilon*, *img_mask=None*)

Perform the iterative AIA algorithm (for PSI) to uncover the phase and phase-steps.

Parameters:

I
[np.array(T,Nx,Ny,M)] T unrelated stacks of M interferograms

delta
[np.array(T,M)] T unrelated sets of initial guesses for delta.

epsilon
[float] Convergence parameter representing the maximum incremental change between delta values.

img_mask
[np.array(T,Nx,Ny)] Defines a mask of valid pixels, so as to allow differently sized images to be processed together. Intensity array I, should be zero-padded where pixels are invalid. Value of None implies all pixels are valid. Default None.

Returns:

phi
[np.array(T,Nx,Ny)] Final calculated phase map.

delta
[np.array(T,M)] Final phase-shift mapping.

n_iter
[np.array(T)] Number of iterations that each batch/test (t in 1,2,…,T) took.

Citations:
[5] [6]

FDA(*step_size*, *lambda0*, *n=1*, *ng=1*, *n_wavenumbers=3*)

Performs Frequency Domain Analysis on the provided interferogram stack, I with step_size delta between adjacent scans.

Parameters:

I
[np.array(Nx,Ny,M)] Stack of M interferograms.

step_size
[float] The step size between adjacent scans, in microns.

lambda0
[float] Central wavelength of the light source, in microns.

n
[float (default 1.0)] index of refraction of sampled material

ng
[float (default = 1.0)] group velocity index

n_wavenumbers
[int (default = 3)] The number of seperate wavenumbers that are used to evaluate the initial height estimate G0.

Returns:

surface
[np.array(Nx,Ny)] Phase-corrected surface height.

z_est
[np.array(Nx,Ny)] Initial surface height estimate, based off of the group-velocity index G0.

Citations:
[4]

MAX_ITER = 250

static Window(*I*, *N=5*)

Returns the interferogram windowed into seperate groups, I1, I2, …, IN

Parameters:

I
[np.array(…, M)] Interfergram where the last parameter is the scan index

Returns:

***Ik, k=1,2,…,N np.array(…,M-N+1)**
Shifted intensity maps.

class OptMetTools.Interferogram.generate.**Generate**

Bases: object

Interferogram.Generate module has methods useful for generating interferograms

`static PSI`(*Z*, *scanning_steps*, *A=0.5*, *B=0.5*, *noise=0*, *phase_offset=0*)

Generates a PSI interferogram stack. Input height is in Radians.

Parameters:

Z
[np.array(Nx,Ny)] Input surface.

scanning_steps
[np.array(Nz)] Array representing the height (or phase) of each scan.

A
[float] Background illumnination constant.

B
[float] Modulation contrast constant.

Noise
[float in range [0,1]] The amount of random noise to be added to the interferograms. Noise is measured as the variance (sigma) as a fraction over parameter B (Noise = sigma/B).

phase_offset
[float] Initial phase offset (radians).

Returns:

I
[np.array(Nx,Ny,Nz)] Stack of interferograms

`static WLI`(*Z*, *scanning_steps*, *lambda0*, *Lc*, *A=0.5*, *B=0.5*, *noise=0*, *phase_offset=0*)

Generates a white-light interferogram stack. Input height is in objective units (i.e., Micron).

Parameters:

Z
[np.array(Nx,Ny)] Input surface.

scanning_steps
[np.array(Nz)] Array representing the height (or phase) of each scan.

lambda0
[float] Central wavelength of light.

Lc
[float] Coherence length.

A
[float] Background illumnination constant. Default 0.5.

B
[float] Modulation contrast constant. Default 0.5

Noise
[float in range [0,1]] The amount of random noise to be added to the interferograms. Noise is measured as the variance (sigma) as a fraction over parameter B (Noise = sigma/B).

phase_offset
[float] Initial phase offset (radians).

Returns:

I
[np.array(Nx,Ny,Nz)] Stack of interferograms

CHAPTER

TWO

SURFACE

class OptMetTools.Surface.generate.**Generate**

Bases: object

The Surface.Generate module is a collection of methods that generate (simulate) different kinds of surfaces.

static Gaussian(*Nx=64*, *Ny=None*, *convex=True*, *x0=None*, *y0=None*, *height=1*, *sigma=32*, *offset=0*)

Returns a gaussian surface. Surfaces Z-values always range from (0,height).

Parameters:

Nx
[int] Number of pixels in the X-dimension. Default is 64.

Ny
[int *optional*] Number of pixels in the Y-Dimenion. If Ny is None, Ny will be set to Nx.

convex
[bool] Determines whether the surface curves inward [False] (concave), or outward [True] (convex).

height
[float] The overall height of the surface. Default: 1

offset
[float] Constant offset added to the height of the surface.

Returns

Z
[np.array(Nx,Ny)] The resulting map of surface height.

static Paraboloid(*Nx=64*, *Ny=None*, *convex=True*, *height=1*, *offset=0*)

Returns a paraboloid surface. Surfaces Z-values always range from (0,height).

Parameters:

Nx
[int] Number of pixels in the X-dimension. Default is 64.

Ny
[int *optional*] Number of pixels in the Y-Dimenion. If Ny is None, Ny will be set to Nx.

convex
[bool] Determines whether the surface curves inward [False] (concave), or outward [True] (convex).

height
[float] The overall height of the surface. Default: 1

offset
[float] Constant offset added to the height of the surface.

Returns

Z
[np.array(Nx,Ny)] The resulting map of surface height.

`static Peaks`(*N=49*, *height=1*, *offset=0*)

Returns a MatLab-like Peaks surface. Surfaces Z-values always scaled from (0,height).

Parameters:

N
[int] Number of pixels in the X- and Y-Dimensions. Default is 49.

height
[float] The overall height of the surface. Default: 1

offset
[float] Constant offset added to the height of the surface.

Returns

Z
[np.array(Nx,Ny)] The resulting map of surface height.

`static Planar`(*Nx=64*, *Ny=None*, *tilt_direction=0*, *height=1*, *offset=0*)

Returns a planar surface. Surfaces Z-values always range from (0,height).

Parameters:

Nx
[int] Number of pixels in the X-dimension. Default is 64.

Ny
[int *optional*] Number of pixels in the Y-Dimenion. If Ny is None, Ny will be set to Nx.

tilt_direction
[int] Angle in degrees [0,360) that controls which edge of the part is highest (i.e., the tilt direction). Common values: - 0 → right edge is highest - 90 → top edge is highest - 180 → left edge is highest - 270 → bottom edge is highest Default: 0

offset
[float] Constant offset added to the height of the surface.

height
[float] The overall height of the surface. Default: 1

Returns:

Z
[np.array(Nx,Ny)] The resulting map of surface height.

`class OptMetTools.Surface.analyze.Analyze`

Bases: `object`

`elliptic_paraboloid`(*x0*, *y0*, *theta*, *a*, *b*, *c*)

returns an elliptic paraboloid on inputs X,Y with fitted parameters.

Parameters:

meshgrids
[np.array(2, nx, ny)] X and Y inputs, stacked on the first dimension

x0
[float] center of paraboloid (X)

y0
[float] center of paraboloid (Y)

theta
[float] Rotation parameter

a
[float] scaling parameter

b
[foat] scaling parameter

c
[float] offset parameter

Returns:

np.array(nx,ny)
surface cosntructed according to input meshgrids & parameters.

Notes:
uses the following model

$$Z(x,y) = \frac{1}{a^2}(X-x)\cos\theta + (Y-y)\sin\theta)^2 + \frac{1}{b^2}((Y-y)\cos\theta - (X-x)\cos\theta)$$

`fit_elliptic_paraboloid`(*target*, *x0*, *y0*, *uncertainty=None*)

Fits an elliptic paraboloid to target data.

Parameters:

predictor
[np.array(2, nx, ny)] Supplied input/independent variables. predictor[0] is x axis, and predictor[1] is y-axis.

target
[np.array(nx,ny)][np.array(nx, ny)] target/dependent variable to fit to.

x0
[float] initial index of center of paraboloid (X)

y0
[float] initial index of center of paraboloid (Y)

uncertainty
[float or None] array of weights in shape of target variable, when performing fit.

Returns:
coefficients

`static fit_gaussian_1d`(*predictor*, *target*)

Fit's a 1D-Guassian Curve using Scipy Optimize Curvefit.

Parameters:

predictor
[np.array(N)] Input tensor (independent variables, etc)

target
[np.array(N)] Output tensor (dependent variables, etc)

static gaussian1d(*x*, *A*, *mu*, *sigma*, *offset*)

Returns a gaussian evaluated at each value in x according to the supplied parameters.

Parameters:

x
[np.array(N)] Supplied input tensor.

A
[float] Amplitude

mu
[float] mean

sigma
[float] variance

offset
[float] constant offset

class OptMetTools.Surface.utils.**Utils**

Bases: object

circular_mask(*ny*, *cx*, *cy*, *r*)

Generates a circular mask of radius *r* about center points (cx, cy) for an image of size (nx,ny).

meshgrid(*ny*, *dtype=None*)

Returns a meshgrid with simply the size parameters

read_ascii()

Reads in an ASCII tab-seperated topographic map and returns a NumPy meshgrid of the surface.

Parameters:

filepath
[str] Absolute or relative path to file

Returns:

tuple(np.array(nx,ny), np.array(nx,ny), np.array(nx,ny)
Returns the X, Y meshgrid axes with the real-valued coordaintes as well as the Z array.

regression()

Performs a regression on all non NaN data points of Z, and returns the plane of best fit.

Parameters:

Z
[np.array(nx,ny)] Input surface

Returns:

Plane
[np.array(nx,ny)] Plane of best fit

INDEX

A

AIA() (*OptMetTools.Interferogram.process.Process static method*), 1
Analyze (*class in OptMetTools.Surface.analyze*), 5

C

circular_mask() (*OptMetTools.Surface.utils.Utils method*), 7

E

elliptic_paraboloid() (*OptMetTools.Surface.analyze.Analyze method*), 5

F

FDA() (*OptMetTools.Interferogram.process.Process method*), 2
fit_elliptic_paraboloid() (*OptMetTools.Surface.analyze.Analyze method*), 6
fit_gaussian_1d() (*OptMetTools.Surface.analyze.Analyze static method*), 6

G

Gaussian() (*OptMetTools.Surface.generate.Generate static method*), 4
gaussian1d() (*OptMetTools.Surface.analyze.Analyze static method*), 6
Generate (*class in OptMetTools.Interferogram.generate*), 2
Generate (*class in OptMetTools.Surface.generate*), 4

M

MAX_ITER (*OptMetTools.Interferogram.process.Process attribute*), 2
meshgrid() (*OptMetTools.Surface.utils.Utils method*), 7

P

Paraboloid() (*OptMetTools.Surface.generate.Generate static method*), 4
Peaks() (*OptMetTools.Surface.generate.Generate static method*), 5
Planar() (*OptMetTools.Surface.generate.Generate static method*), 5
Process (*class in OptMetTools.Interferogram.process*), 1
PSI() (*OptMetTools.Interferogram.generate.Generate static method*), 2

R

read_ascii() (*OptMetTools.Surface.utils.Utils method*), 7
regression() (*OptMetTools.Surface.utils.Utils method*), 7

U

Utils (*class in OptMetTools.Surface.utils*), 7

W

Window() (*OptMetTools.Interferogram.process.Process static method*), 2
WLI() (*OptMetTools.Interferogram.generate.Generate static method*), 3